

TEMA 2 → INTRODUCCIÓN A LA VIRTUALIZACIÓN

1. Introducción a la virtualización

La **virtualización** es una tecnología que permite crear entornos virtuales sobre un sistema físico. Tradicionalmente, se ha usado para crear máquinas virtuales (VM) que ejecutan sistemas operativos completos sobre un hipervisor, aprovechando al máximo los recursos de hardware y facilitando la gestión y el aislamiento de aplicaciones.

Con el avance de la tecnología y las necesidades de eficiencia, han surgido alternativas como los **contenedores**, que permiten virtualizar a nivel de sistema operativo, compartiendo el kernel y ofreciendo un entorno aislado y ligero para ejecutar aplicaciones.

2. Introducción a Docker y Conceptos Fundamentales

¿Qué es Docker?

Docker es una plataforma de código abierto que permite construir, empaquetar y ejecutar aplicaciones dentro de contenedores. Estos contenedores incluyen todo lo necesario para que la aplicación funcione igual en cualquier entorno, haciendo más fácil el desarrollo, la distribución y el despliegue. Fue lanzada en 2013

¿Qué es un contenedor?

Un contenedor es como una “caja” donde metes tu aplicación con todo lo que necesita para funcionar, permitiendo que se ejecute igual en cualquier ordenador. Es ligero, rápido y mantiene la aplicación aislada del resto del sistema.

¿Qué es OCI?

OCI (Open Container Initiative) es un proyecto de estándar abierto fundado en 2015 para definir especificaciones sobre la creación, ejecución y distribución de contenedores. Su objetivo es garantizar la compatibilidad entre diferentes herramientas, como Docker, Podman, y runtimes como containerd y runc.

Otros conceptos clave

- **Imagen:** Archivo inmutable compuesto por capas, que contiene el sistema de archivos y dependencias para crear contenedores.
- **Registro:** Repositorio centralizado donde se almacenan y comparten imágenes (ej: Docker Hub).
- **Orquestación:** Conjunto de herramientas para automatizar la gestión de múltiples contenedores (ej: Docker Compose, Kubernetes).
- **Microservicio:** Arquitectura que descompone aplicaciones en servicios pequeños e independientes, facilitando el uso de contenedores.

3. Contenedores vs. Máquinas Virtuales

Característica	Contenedores	Máquinas Virtuales
Virtualización	OS-level (núcleo compartido)	Hardware-level (núcleo propio)
Peso y recursos	Ligeros, MB	Pesadas (GB)
Arranque	Segundos	Minutos
Portabilidad	Alta	Limitada
Aislamiento	Muy bueno, depende del kernel	Absoluto (SO y hardware)

- Contenedores: ideales para microservicios, CI/CD, desarrollo y pruebas.
 - Máquinas Virtuales: útiles en sistemas legacy y máxima seguridad.
-

4. Arquitectura y Componentes Docker

- **Docker Engine:** Daemon central que gestiona imágenes, contenedores, redes y volúmenes.
→ *Es el “motor” de Docker que hace que todo funcione en tu ordenador o servidor.*
- **CLI (Command Line Interface):** Interfaz de línea de comandos para operar Docker.
→ *Es como hablarle a Docker con comandos en la terminal.*
- **Imágenes:** Plantillas de solo lectura para crear contenedores.
→ *Son como recetas que dicen cómo construir un contenedor.*
- **Contenedores:** Instancias vivas y aisladas de una imagen.
→ *Son los contenedores “reales” que corren tu aplicación.*
- **Registro (Registry):** Repositorios como Docker Hub para compartir imágenes.
→ *Es como una biblioteca donde guardas y descargas las imágenes.*
- **Volúmenes:** Para datos persistentes.
→ *Son carpetas que guardan información aunque el contenedor se borre.*
- **Redes:** Comunicación segura entre contenedores.
→ *Son los canales que permiten que varios contenedores hablen entre sí.*

5. Dockerfile

Un **Dockerfile** es un archivo de texto donde escribes **instrucciones para construir una imagen de Docker**.

En otras palabras, es como una **receta** que le dice a Docker:

- Qué sistema base usar
- Qué programas instalar
- Qué archivos copiar
- Qué comando ejecutar cuando el contenedor se inicie

6. Docker Compose

Docker Compose es una herramienta que permite **definir y ejecutar aplicaciones con múltiples contenedores** usando un solo archivo de configuración llamado `docker-compose.yml`.

En otras palabras:

➡ Si tu aplicación necesita, por ejemplo, un **servidor web** y una **base de datos**, Docker Compose te permite levantar **los dos contenedores a la vez** con un solo comando.

Cómo funciona

- Se escribe un archivo `docker-compose.yml` donde se describen todos los servicios, imágenes, puertos, volúmenes y redes que usarán los contenedores.
- Con un solo comando, como `docker-compose up -d`, Docker Compose crea y ejecuta **todos los contenedores configurados** automáticamente.

Ejemplo docker-compose.yml :

yaml

```
version: '3.8'

services:
  mysql-db:
    image: mysql:8.0
    container_name: mysql-db
    environment:
      MYSQL_ROOT_PASSWORD: mi_clave_mysql
      MYSQL_DATABASE: mi_base_mysql
      MYSQL_USER: usuario_mysql
      MYSQL_PASSWORD: clave_usuario_mysql
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_PASSWORD: mi_clave_postgres
      POSTGRES_DB: mi_base_postgres
      POSTGRES_USER: usuario_postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  mysql_data:
  postgres_data:
```

- **mysql-db**
 - Levanta un contenedor con MySQL 8.0
 - Define usuario, contraseña y base de datos
 - Expone el puerto 3306 para conectarte desde tu PC
 - Guarda los datos en un volumen llamado mysql_data
- **postgres-db**
 - Levanta un contenedor con PostgreSQL 15
 - Define usuario, contraseña y base de datos
 - Expone el puerto 5432
 - Guarda los datos en un volumen llamado postgres_data
- **volumes:**
 - Permiten que los datos de las bases se **mantengan, aunque los contenedores se eliminen**